

■ ANALISIS ARTIKEL JURNAL “A THREE-STAGE ADAPTIVE TASK SCHEDULING STRATEGY FOR LOAD BALANCING IN CLOUD COMPUTING”

KELOMPOK 10

CALISTA SAHLA
24060123130083

ARSYAD GRANT S
24060122140143

FEBRIANTI PUJI
24060123120034

TSABITA SYAHIDA K
24060123130071



LATAR BELAKANG MASALAH

Cloud computing memanfaatkan virtualisasi untuk menyediakan sumber daya yang fleksibel dan skalabel, tetapi keterbatasan resource dan kebutuhan pengguna yang heterogen menimbulkan tantangan besar dalam task scheduling. Tujuan task scheduling adalah mengalokasikan tugas ke virtual machine (VM) secara efisien agar kualitas layanan (QoS) dan load balancing terjaga, serta biaya operasional dapat diminimalkan.

Namun, kinerja penjadwalan masih sering menurun akibat fluktuasi beban kerja dan heterogenitas tugas. Metode yang sudah ada, baik heuristik, hybrid, maupun DRL, masih memiliki kelemahan seperti hasil yang suboptimal, computational overhead yang tinggi, masalah cold-start, serta belum efektif dalam menjaga keseimbangan pemanfaatan resource dan task-VM matching.

Berdasarkan kondisi tersebut, penulis mengusulkan three-stage adaptive task scheduling strategy yang dirancang untuk bekerja tanpa historical data pada tahap awal, meningkatkan akurasi saat data historis tersedia, dan mengatasi runtime load imbalance pada lingkungan cloud yang dinamis.



Cloud computing digunakan untuk menangani banyak tugas secara bersamaan dengan memanfaatkan Virtual Machine [VM]. Namun, dalam praktiknya, proses pembagian tugas [task scheduling] masih belum berjalan secara optimal. Hal ini menyebabkan terjadinya ketidakseimbangan beban kerja antar VM, di mana beberapa VM bekerja terlalu berat sementara yang lain tidak dimanfaatkan secara maksimal. Kondisi ini secara langsung berdampak pada penurunan performa sistem secara keseluruhan.

MASALAH YANG MUNCUL :

- **TASK TIDAK DIALOKASIKAN SECARA EFISIEN**
- **TERJADI LOAD IMBALANCE ANTAR VM**
- **PEMANFAATAN RESOURCE BELUM MAKSIMAL**

ANALISA PERMASALAHAN



PENYEBAB PERMASALAHAN

BEBERAPA PENYEBAB UTAMA:

- **HETEROGENITAS TASK DAN RESOURCE**
- **PERUBAHAN WORKLOAD SECARA DINAMIS**
- **KETERBATASAN KAPASITAS VM**
- **TIDAK ADANYA DATA HISTORIS DI AWAL**

Permasalahan dalam task scheduling ini tidak terjadi begitu saja, tetapi dipengaruhi oleh berbagai faktor dalam lingkungan cloud yang kompleks dan dinamis. Salah satu faktor utama adalah adanya perbedaan karakteristik setiap task, di mana setiap task memiliki kebutuhan resource yang berbeda, seperti CPU, memori, maupun I/O. Selain itu, kondisi workload yang terus berubah membuat sistem sulit menentukan pembagian tugas yang konsisten.

Faktor lain yang memperparah kondisi ini adalah keterbatasan resource pada VM serta tidak adanya data historis pada tahap awal [cold-start problem], sehingga sistem tidak memiliki referensi untuk melakukan penjadwalan yang lebih akurat.



DAMPAK PERMASALAHAN

Akibat dari berbagai permasalahan tersebut, sistem cloud mengalami penurunan efisiensi dalam menjalankan tugas. Waktu penyelesaian task menjadi lebih lama karena adanya ketidaksesuaian antara task dan VM yang digunakan. Selain itu, ketidakseimbangan beban juga menyebabkan beberapa resource bekerja secara berlebihan, sementara yang lain tidak digunakan secara optimal, sehingga meningkatkan biaya operasional dan konsumsi energi.

Secara keseluruhan, permasalahan utama dalam cloud computing terletak pada proses task scheduling yang belum efisien dan belum mampu beradaptasi dengan kondisi sistem yang dinamis. Oleh karena itu, diperlukan pendekatan yang lebih cerdas dan adaptif untuk mengatasi permasalahan tersebut.

DAMPAK YANG TERJADI:

- **WAKTU EKSEKUSI MENINGKAT (MAKESPAN TINGGI)**
- **BIAYA KOMPUTASI BERTAMBAH**
- **LOAD ANTAR VM TIDAK SEIMBANG**
- **KUALITAS LAYANAN (QOS) MENURUN**



SOLUSI MASALAH

TAHAP 1 : SCHEDULING TANPA HISTORICAL DATA [COLD-START SOLUTION]

Pada tahap awal, sistem belum memiliki data historis, sehingga penjadwalan dilakukan berdasarkan:

- kondisi real-time VM [CPU, memori, storage]
- kebutuhan resource dari setiap task

Solusi yang diberikan:

- Menggunakan resource-aware scheduling
- Task dialokasikan ke VM yang paling sesuai berdasarkan kapasitas yang tersedia
- Mengurangi kesalahan alokasi di awal sistem berjalan

Hasil:

- Mengatasi cold-start
- Meningkatkan efisiensi awal sistem
- Mendukung peningkatan system stability

TAHAP 2 : SCHEDULING BERBASIS HISTORICAL DATA [OPTIMASI AKURASI]

Setelah berjalan, data historis mulai dimanfaatkan untuk meningkatkan kualitas scheduling.

Solusi yang diterapkan:

- Task classification → task dibagi menjadi: Compute-Intensive [CI], I/O-Intensive [IO], Hybrid [HY]
- VM classification → VM dikelompokkan berdasarkan kemampuan resource
- Task-VM matching → mencocokkan task dengan VM yang paling sesuai

Hasil:

- Akurasi meningkat [85–92% matching]
- Makespan menurun hingga $\pm 26\%$ lebih baik
- QoS meningkat

TAHAP 3 : ADAPTIVE TASK MIGRATION [LOAD BALANCING DINAMIS]

Untuk mengatasi perubahan workload yang dinamis, sistem melakukan migrasi task secara adaptif.

Solusi yang diberikan:

- Monitoring kondisi VM secara berkala
- Memindahkan task dari VM yang overload ke VM yang lebih ringan
- Mempertimbangkan: biaya migrasi, beban VM, efisiensi eksekusi

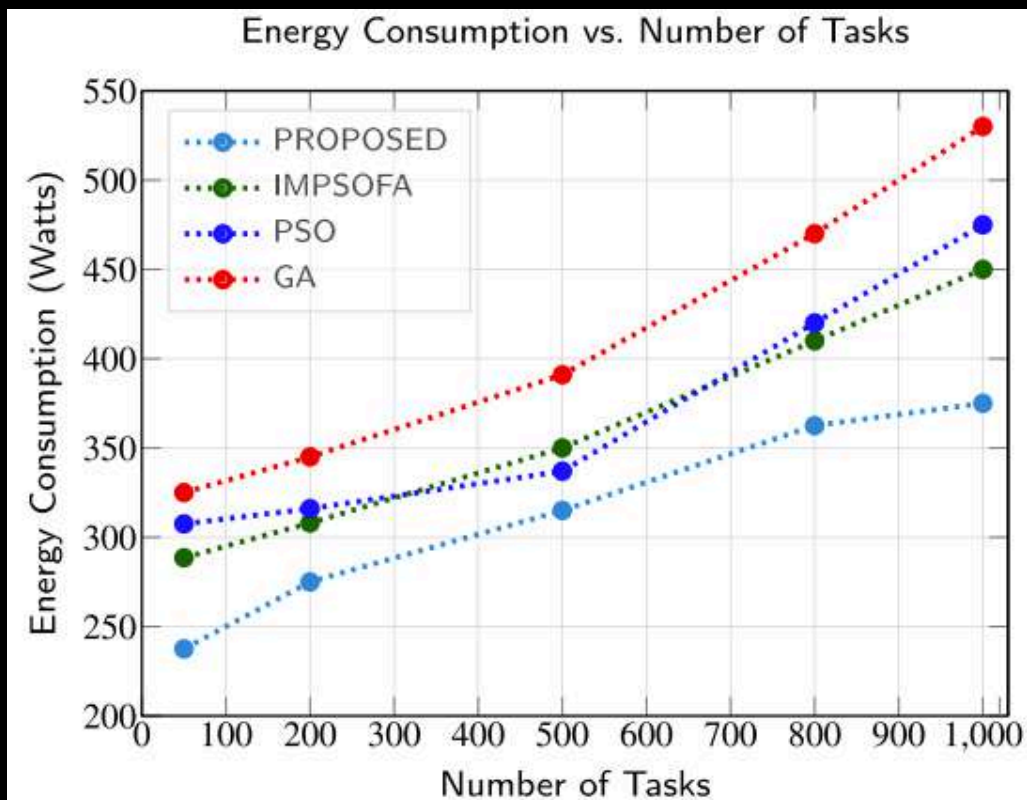
Hasil:

- Load lebih seimbang
- Efisiensi energi meningkat [hingga 29%]
- Resource utilization naik
- Sistem lebih stabil [+30–35%]

HASIL PENELITIAN

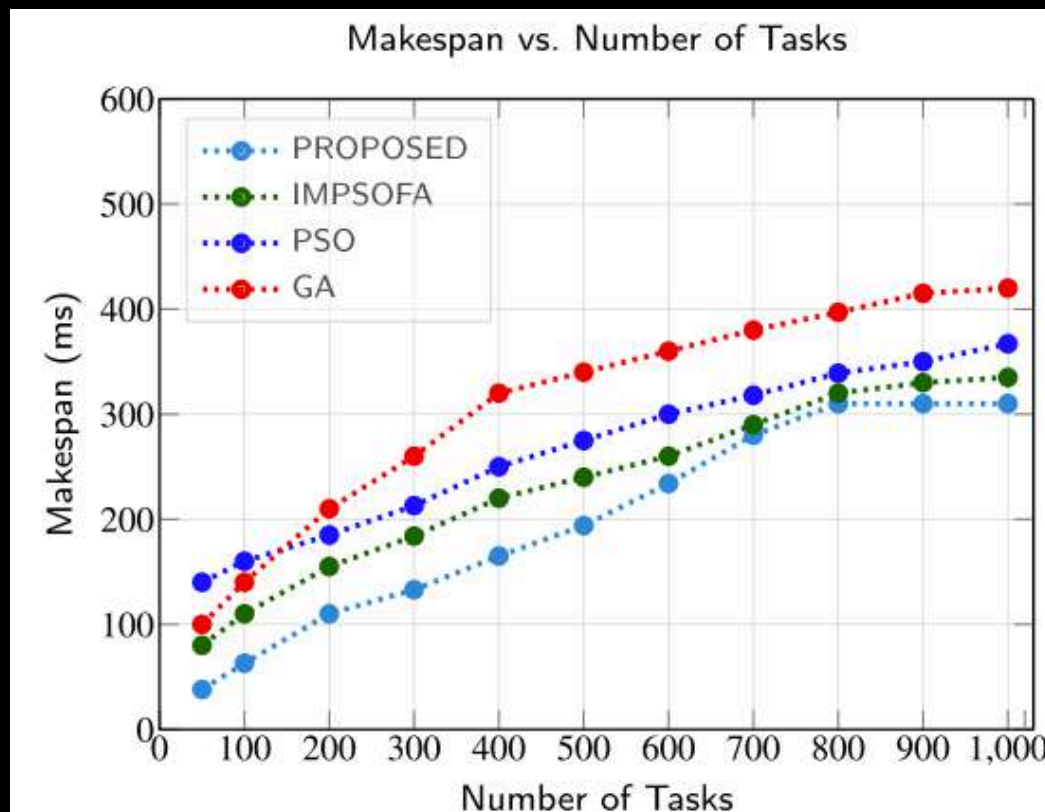
PERBANDINGAN KINERJA DENGAN ALGORITMA BASELINE (IMPSOFA, PSO, & GA)

ENERGY CONSUMPTION



Efisiensi energi meningkat, three-stage mencapai pengurangan konsumsi energi 29.25% dibanding GA, 21.05% dibanding PSO, dan 16.67% dibanding IMPSOFA.

MAKESPAN



Pengurangan makespan yang signifikan dibandingkan algoritma baseline. Hasil menunjukkan algoritma three-stage 26.19% lebih baik dari GA, 15.53% lebih baik dari PSO, dan 7.46% lebih baik dari IMPSOFA.

MATCHING ACCURACY

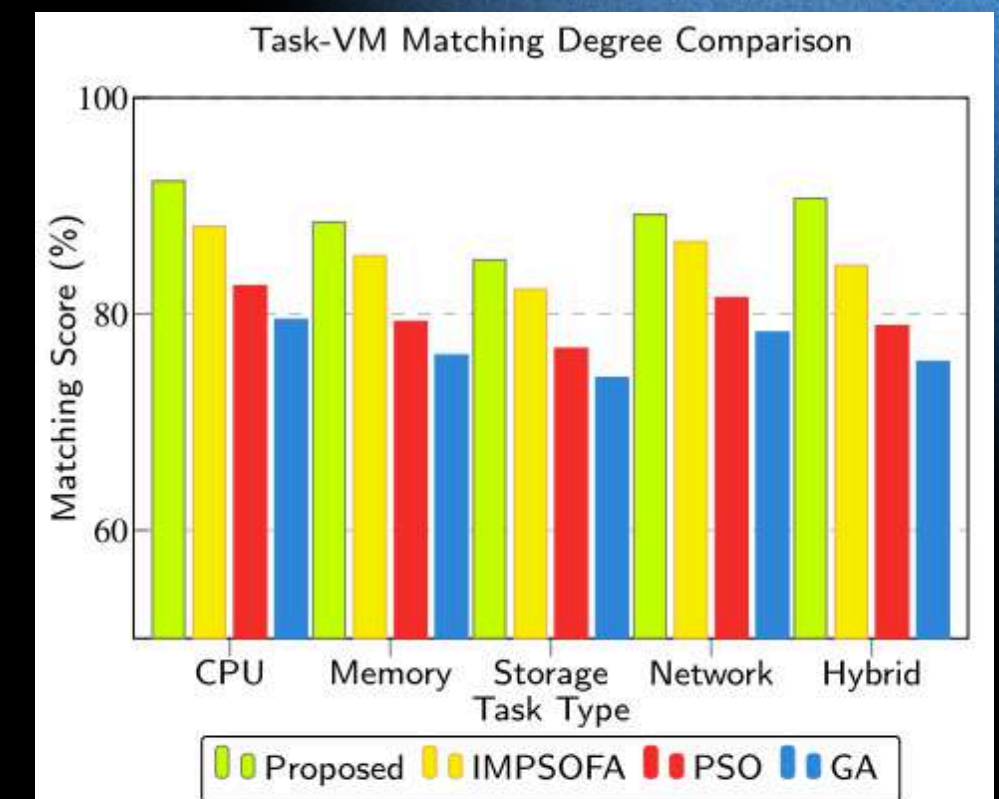


Fig. 10. Comparison of task type to VM type matching.

Three-stage superior dalam Task-VM matching accuracy dengan matching scores konsisten di 85-92%.

HASIL PENELITIAN

PERBANDINGAN KINERJA KETIGA TAHAP

Table 10 Three-stage scheduling algorithm performance comparison.							
Strategy	Makespan (s)	Avg. Delay (ms)	Task Starvation (wait>50ms)	VM Overload Count (load>87.5%)	Peak Load (%)	Min Load (%)	Scheduler Cost (ms)
Random	482 ± 15.2	122 ± 8.3	76 ± 5.1	13 ± 8	vm03(97.4%)	vm24(57.5%)	76.4 ± 4.2ms
Stage-1 Only	390 ± 12.1	87 ± 6.7	42 ± 3.9	6 ± 4	vm05(91.2%)	vm20(75.4%)	51.0 ± 3.9ms
Stage-2 Only	350 ± 9.8	70 ± 5.2	34 ± 2.4	5 ± 4	vm22(90.6%)	vm23(80.6%)	48.2 ± 3.3ms
Stage-1 + Stage-2	310 ± 6.4	66 ± 4.6	19 ± 1.8	3 ± 3	vm04(90.1%)	vm18(73.8%)	37.2 ± 2.7ms
Stage-1 + Stage-2 + Stage-3	310 ± 6.4	57 ± 4.1	8 ± 1.2	0 ± 0	vm12(86.5%)	vm06(78.3%)	28.5 ± 2.1ms

Note: Data shown as mean ± standard deviation over 12 runs. Peak/Min Load show the highest and lowest CPU utilization.

Perbandingan ketiga tahapan tersebut menunjukkan peningkatan bertahap dalam nilai Makespan dan pemanfaatan VM, serta memvalidasi kontribusinya terhadap penjadwalan yang optimal. Hasilnya menunjukkan bahwa setiap tahap memberikan kontribusi signifikan terhadap peningkatan kinerja. Hal ini menunjukkan bahwa stage 2 secara signifikan meningkatkan scheduling accuracy melalui historical knowledge, sementara stage 3 melakukan fine-tuning alokasi melalui adaptive migration, terutama di bawah dynamic load conditions. Kombinasi ketiga tahap menghasilkan peningkatan sekitar 30-35% pada system stability, 15-20% pada utilization, dan 10-12% pada energy efficiency dibandingkan single-stage scheduling.

ANALISA IMPLEMENTASI CLOUD COMPUTING

IMPLEMENTASI:

- Berbasis IaaS (Virtual Machine)
- Menggunakan 34 VM (HPC/HT/BC)
- Menggunakan simulator CloudSim
- Scheduler sebagai pusat kontrol (3-stage adaptive)
- Lingkungan heterogen & dinamis
- Menggunakan workflow (DAG) + monitoring real-time
- Mendukung task migration

KELEBIHAN:

- Adaptif terhadap perubahan load
- Mengatasi cold-start problem
- Resource-aware (CPU, I/O, memori)
- Hybrid approach (heuristic + data historis + migration)

KEKURANGAN:

- Tidak mempertimbangkan network latency
- Overhead migration belum detail
- Masih berbasis VM (belum container)
- Energy model sederhana



THANK YOU